

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability. (BL3, BL4)*

Activity Tool : Mock Interview (Low)
Assessment Tool Weightage: 30%

Dr

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'	BL3 – Apply	5
2	Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'	BL4 – Analyze	5
3	Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'	BL3 – Apply	5
4	Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'	BL3 – Apply	5
5	Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'	BL4 – Analyze	5
6	Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'	BL4 – Analyze	5
7	Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'	BL4 – Analyze	5

8	Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'	BL3 – Apply	5
9	Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'	BL4 – Analyze	5
10	Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'	BL4 – Analyze	5

Code of Conduct:

I Geferlin Boffy R 192571080 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated

Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

1. Interviewer: Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?

Answer:

Query processing is the process of converting an SQL query into an efficient execution plan and producing the required result. The major steps are:

1. Parsing and Syntax Analysis

The SQL query is first analyzed by the parser. It checks whether the query follows the correct SQL syntax and generates a **parse tree**.

2. Semantic Analysis

The DBMS verifies the meaning of the query. It checks whether the referenced tables and columns exist, whether the data types are compatible, and whether the user has the required permissions.

3. Query Translation

The parsed SQL query is converted into an equivalent **relational algebra expression or logical query plan**.

4. Query Optimization

The query optimizer generates different possible execution plans. It considers different methods for:

- Table scanning
- Index usage
- Join algorithms
- Join ordering
- Sorting and grouping

The optimizer estimates the cost of each plan using database statistics such as table size, number of tuples, available indexes, selectivity, memory, CPU usage, and mainly **I/O cost**.

5. Selection of Best Plan

The optimizer compares the estimated costs and selects the execution plan with the lowest estimated cost.

6. Query Execution

The selected physical execution plan is given to the query execution engine. The required operations are performed and data is retrieved from memory or disk.

7. Result Generation

The final result is generated and returned to the user or application.

Flow:

SQL Query → Parsing → Semantic Analysis → Translation → Optimization → Execution Plan → Execution → Result

Thus, the query optimizer improves performance by selecting the execution plan that is expected to require the least resources and execution time.

2. Interviewer: What is the difference between heuristic optimization and cost-based optimization? When would you use each?

Answer:

Query optimization is the process of selecting an efficient execution plan for a query. Two important approaches are **heuristic optimization** and **cost-based optimization**.

Heuristic Optimization

Heuristic optimization uses predefined rules or transformations to improve a query without calculating the exact execution cost of every possible plan.

Common heuristic rules are:

1. Perform **selection as early as possible**.
2. Perform **projection as early as possible**.
3. Reduce the number of tuples before performing joins.
4. Choose efficient join operations.
5. Use indexes whenever appropriate.

For example, if a query contains a selection condition before a join, the DBMS may apply the selection first to reduce the number of tuples participating in the join.

Cost-Based Optimization

Cost-based optimization generates multiple possible execution plans and estimates the cost of each plan.

The cost may include:

- Disk I/O
- CPU processing
- Memory usage
- Number of tuples processed

- Join cost
- Index access cost

The plan with the lowest estimated cost is selected.

Heuristic Optimization

Uses predefined rules

Faster optimization

Simple approach

Does not require detailed statistics

May not always produce the best plan

Cost-Based Optimization

Uses estimated execution costs

Requires more optimization time

More complex approach

Requires database statistics

Usually produces a more efficient plan

When to Use

Heuristic optimization is useful for simple queries, when quick optimization is required, or when detailed statistics are unavailable.

Cost-based optimization is preferred for complex queries involving multiple joins, indexes, sorting, and large tables because it can compare different execution plans and select the least expensive one.

Therefore, modern DBMSs generally prefer **cost-based optimization**, often combined with heuristic rules.

3. Interviewer: How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.

Answer:

Database connectivity allows an application to communicate with a database for storing, retrieving, updating, and deleting data.

Steps to Implement Database Connectivity

1. **Install the database and driver** required for the application.
2. **Configure database details** such as database URL, username, and password.
3. **Establish a connection** between the application and database.
4. **Create a statement or prepared statement** for executing SQL queries.
5. **Execute the SQL query.**
6. **Process the result set** returned by the database.
7. **Commit the transaction** if the operation is successful.
8. **Rollback the transaction** if an error occurs.
9. **Close or return the connection** after completing the operation.

For large applications, a **connection pool** can be used. It maintains a group of reusable database connections, reducing the overhead of creating a new connection for every request.

JDBC

JDBC stands for Java Database Connectivity. It is a standard Java API used to connect Java applications with relational databases.

ODBC

ODBC stands for Open Database Connectivity. It is a standard interface that allows applications to communicate with databases using database-specific ODBC drivers.

JDBC

Java-specific API

Mainly used by Java applications

Uses JDBC drivers

Provides Java-based database access

Commonly used in Java applications

ODBC

Language-independent API

Used by applications in different languages

Uses ODBC drivers

Provides general database connectivity

Commonly used for cross-language database access

Example: In a Java banking application, JDBC can be used to connect the application to a MySQL database, execute transactions, and retrieve account information.

4. Interviewer: What are ACID properties? Give a real-world example where each property is critical.

Answer:

ACID stands for **Atomicity, Consistency, Isolation, and Durability**. These properties ensure that database transactions are reliable, correct, and safe even when multiple transactions execute simultaneously or system failures occur.

1. Atomicity

Atomicity means that a transaction is treated as a **single indivisible unit**. Either all operations are completed or none of them are performed.

Example:

Consider a bank transfer of ₹5,000 from Account A to Account B.

- ₹5,000 must be deducted from A.
- ₹5,000 must be added to B.

If adding the money to B fails after deducting it from A, the deduction must also be rolled back.

2. Consistency

Consistency ensures that a transaction changes the database from **one valid state to another valid state** while maintaining all integrity constraints.

Example:

In a banking system, the database must maintain valid account balances and should not violate rules such as invalid account numbers or transaction constraints.

3. Isolation

Isolation ensures that concurrent transactions do not interfere with each other. Each transaction should behave as if it is executing independently.

Example:

If two customers try to book the same seat simultaneously, isolation ensures that only one transaction successfully reserves the seat.

4. Durability

Durability means that once a transaction is committed, its changes are **permanently stored** and should survive system crashes or power failures.

Example:

After an online payment is successfully completed, the payment record must remain stored even if the server crashes immediately afterward.

Summary

Atomicity → **All or Nothing**

Consistency → **Valid State to Valid State**

Isolation → **Transactions Do Not Interfere**

Durability → **Committed Data Persists**

5. Interviewer: Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?

Answer:

Concurrency control is the process of managing simultaneous transactions so that database consistency and isolation are maintained.

1. Dirty Read

A dirty read occurs when one transaction reads data that has been modified by another transaction but has **not yet been committed**.

Example:

- T1 changes balance from ₹10,000 to ₹8,000.
- T2 reads ₹8,000.
- T1 then rolls back.
- The actual balance becomes ₹10,000.

T2 has read an invalid or uncommitted value.

2. Lost Update

A lost update occurs when two transactions read the same data and update it, causing one transaction's update to overwrite the other.

Example:

- T1 reads balance = ₹1,000.
- T2 reads balance = ₹1,000.

- T1 updates it to ₹1,200.
- T2 updates it to ₹1,100.

T1's update is lost.

3. Unrepeatable Read

An unrepeatable read occurs when a transaction reads the same data item twice and gets different values because another transaction modified and committed the data between the two reads.

Example:

- T1 reads salary = ₹30,000.
- T2 changes salary to ₹35,000 and commits.
- T1 reads the salary again and gets ₹35,000.

How 2PL Solves These Problems

Two-Phase Locking (2PL) divides transaction execution into two phases:

Growing Phase:

The transaction can acquire locks but cannot release them.

Shrinking Phase:

The transaction can release locks but cannot acquire new locks.

Two types of locks are commonly used:

- **Shared Lock (S):** Used for reading.
- **Exclusive Lock (X):** Used for writing.

An exclusive lock prevents other conflicting transactions from reading or modifying the data until the lock is released.

Therefore, 2PL ensures **conflict serializability** and prevents many concurrency problems. **Strict 2PL**, which holds exclusive locks until commit or rollback, additionally prevents dirty reads/writes and cascading rollbacks.

6. Interviewer: What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.

Answer:

A **deadlock** is a situation in which two or more transactions wait indefinitely for resources held by each other. As a result, none of the transactions can proceed.

Example

Suppose:

- T1 holds a lock on data item A and waits for B.
- T2 holds a lock on data item B and waits for A.

Therefore:

T1 → T2 → T1

This cycle represents a deadlock.

Deadlock Detection Using Wait-For Graph

A **Wait-For Graph (WFG)** represents transactions as nodes.

The algorithm is:

1. Create a node for every active transaction.
2. If transaction T_i is waiting for a resource held by T_j , create an edge:

$T_i \rightarrow T_j$

3. Construct the complete Wait-For Graph.
4. Apply a cycle detection algorithm such as DFS.
5. If a cycle exists, a deadlock is detected.
6. Select one transaction as the victim.
7. Roll back the victim transaction and release its locks.
8. Allow the remaining transactions to continue.

Deadlock Prevention Strategies

1. Wait-Die Scheme

It is a timestamp-based method.

- An older transaction can wait for a younger transaction.
- A younger transaction requesting a resource held by an older transaction is rolled back.

2. Wound-Wait Scheme

- An older transaction can force a younger transaction to roll back.
- A younger transaction waits for an older transaction.

3. Timeout

A transaction is rolled back if it waits for a lock longer than a specified time. Thus, deadlock detection identifies cycles after they occur, while prevention strategies attempt to ensure that deadlocks do not occur.

7. Interviewer: Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?

Answer:

Database recovery techniques restore the database to a consistent state after failures. Two important techniques are **Immediate Update** and **Deferred Update**.

Immediate Update

In Immediate Update, changes made by a transaction may be written to the database **before the transaction commits**.

The log stores both:

<Transaction, Data Item, Old Value, New Value>

During recovery:

- **UNDO** is required for uncommitted transactions.
- **REDO** may be required for committed transactions.

Deferred Update

In Deferred Update, changes are not written to the database until the transaction successfully commits. The updates are first recorded in the log or temporary storage.

During recovery:

- Committed transactions are **REDO** operations.
- Uncommitted transactions generally do not require UNDO because their changes were not written to the database.

Immediate Update

Database can be updated before commit

Requires UNDO and REDO

Recovery is more complex

Log stores old and new values

Suitable for systems allowing early page writes

Deferred Update

Database is updated after commit

Mainly requires REDO

Recovery is simpler

Log mainly stores new values

Suitable when simpler recovery is preferred

Preferred Scenarios

Immediate Update is preferred in systems where database pages may need to be written before transaction commit and flexible buffer management is required.

Deferred Update is preferred when simpler recovery is desired and the system can postpone writing database changes until commit.

Therefore, the main difference is **when database modifications are physically written** and consequently which recovery operations are required.

8. Interviewer: Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?

Answer:

Shadow Paging is a database recovery technique that maintains two page tables:

1. **Shadow Page Table**
2. **Current Page Table**

The shadow page table points to the stable version of the database, while the current page table represents the modified version.

Working of Shadow Paging

1. Initially, both page tables point to the same database pages.
2. When a page needs to be modified, the original page is not overwritten.
3. A new page is allocated and the updated data is written to it.
4. The current page table is modified to point to the new page.
5. The shadow page table continues pointing to the original page.
6. If the transaction commits, the current page table becomes the new stable page table.
7. If a crash occurs before commit, the system discards the current page table and uses the shadow page table.

Advantages

1. Recovery is simple and fast.
2. No complex UNDO operation is required.
3. Original database pages remain unchanged until commit.
4. It does not require traditional log processing for recovery.

Limitations

1. Requires additional storage for new page copies.
2. Page table management can become expensive.
3. Can cause fragmentation.
4. Frequent page updates may result in significant copying overhead.
5. It is less suitable for large, highly concurrent databases.
6. Log-based recovery is generally more flexible and scalable for large transaction systems.

Comparison

Shadow Paging → Simple and fast recovery but higher page-management overhead.

Log-Based Recovery → More complex but flexible and suitable for large transaction-processing systems.

9. Interviewer: What is a serializability schedule? How do you test for conflict serializability using a precedence graph?

Answer:

A **schedule** is the sequence in which operations of multiple transactions are executed. A schedule is **serializable** if its final result is equivalent to the result produced by executing the same transactions in some serial order.

A **serial schedule** executes one transaction completely before starting another transaction.

Conflict Serializability

A schedule is conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.

Two operations conflict when:

1. They belong to different transactions.
2. They access the same data item.
3. At least one of them is a write operation.

The conflicting pairs are:

- Read–Write
- Write–Read
- Write–Write

Precedence Graph Algorithm

1. Create one node for every transaction.
2. Scan the schedule from left to right.
3. Identify every pair of conflicting operations.
4. If an operation of T_i occurs before a conflicting operation of T_j , create an edge:

$T_i \rightarrow T_j$

5. Continue until all conflicting operations are considered.
6. Check the graph for cycles.

Result

If the precedence graph has no cycle:

The schedule is **conflict serializable**.

If the precedence graph contains a cycle:

The schedule is **not conflict serializable**.

Example

Suppose the graph is:

$T_1 \rightarrow T_2 \rightarrow T_3$

There is no cycle, so the schedule is conflict serializable. The equivalent serial order is:

$T_1 \rightarrow T_2 \rightarrow T_3$

If the graph is:

$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1$

a cycle exists, so the schedule is not conflict serializable.

Thus, the **absence of a cycle in the precedence graph is the test for conflict serializability**.

10. Interviewer: Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and

Rigorous 2PL. Which is most commonly used and why?

Answer:

Two-Phase Locking (2PL) is a concurrency control protocol that uses locks to ensure conflict serializability. It divides transaction execution into a **growing phase** and a **shrinking phase**.

1. Basic 2PL

Basic 2PL has two phases:

Growing Phase:

- The transaction can acquire locks.
- It cannot release locks.

Shrinking Phase:

- The transaction can release locks.
- It cannot acquire any new locks.

Basic 2PL guarantees conflict serializability but may allow **cascading rollbacks**.

2. Strict 2PL

In Strict 2PL, all **exclusive (X) locks** are held until the transaction commits or aborts.

Advantages:

- Prevents dirty writes.
- Prevents cascading rollbacks.
- Simplifies recovery.
- Ensures conflict serializability.

3. Rigorous 2PL

In Rigorous 2PL, **both shared (S) locks and exclusive (X) locks** are held until the transaction commits or aborts.

It provides stronger isolation than Basic 2PL but can reduce concurrency because even shared locks are held for a longer time.

Comparison

Feature	Basic 2PL	Strict 2PL	Rigorous 2PL
Conflict serializability	Yes	Yes	Yes
X locks held until commit	No	Yes	Yes
S locks held until commit	No	Not necessarily	Yes
Cascading rollback	Possible	Prevented	Prevented
Concurrency	Higher	Good	Lower
Recovery	More complex	Easier	Easier

Most Commonly Used

Strict 2PL is commonly used because it provides a good balance between **concurrency, consistency, and recoverability**. By holding exclusive locks until commit or rollback, it prevents other transactions from reading or overwriting uncommitted data and avoids cascading rollbacks.

Conclusion:

Basic 2PL provides serializability, Strict 2PL additionally provides easier recovery and prevents cascading rollback, while Rigorous 2PL provides the strongest locking discipline by holding all locks until the transaction completes.